LOWKEY — Project Plan & Roadmap

Core Idea: *Discover songs you didn't know you'd love. LOWKEY recommends lesser-known tracks that match your taste while helping you explore beyond your usual music bubble.*

---

1. Decisions Made So Far

- **Project Name:** LOWKEY.
- **Core Experience:** User enters a song and receives recommendations that preserve its vibe while introducing unexpected discoveries.
- **Two Recommendation Signals:**
  1. *Content-based similarity:* Find similar songs using audio features or embeddings.
  2. *Collaborative filtering:* Find songs listened to by people who also listened to the input song.
- **Discovery Layer:** Prioritize lesser-known tracks without sacrificing musical relevance.
- **Data Strategy:** Prefer one static dataset supporting both audio similarity and listening-history analysis, avoiding continuous syncing or dependence on live APIs.

2. Proposed Workflow

- **Step 1:** User enters a song.
- **Step 2:** Retrieve candidate songs using musical similarity and listener overlap.
- **Step 3:** Rank candidates using relevance, popularity/underheard signals, and diversity.
- **Step 4:** Display recommendations with explanations for why each song was selected.
- **Step 5:** Let users refine recommendations using natural language feedback.

3. Dataset Being Evaluated

- **#nowplaying-RS:** A potential single-dataset solution containing historical listening events and audio features.
- **Verification Required:** Verify track-ID-to-song-name mapping, available audio features, coverage, and licensing.
- **Contingency:** If it cannot support both recommendation signals adequately, evaluate alternative datasets.

4. Still To Be Decided

- Final dataset and track-metadata mapping.
- Whether to use traditional audio features, pretrained audio embeddings, or both.
- Collaborative-filtering method and how to combine it with content similarity.
- Metric to measure whether a recommendation is genuinely less mainstream.
- Final ranking formula and fallback for songs missing from the dataset.
- Frontend design and the scope of the three-day MVP.

5. Team Structure & MVP Goal

- **AI / Full-stack:** Data pipeline, recommendation integration, backend APIs, and frontend.
- **ML:** Dataset analysis, feature-based similarity, collaborative filtering, ranking experiments, and evaluation.

- **MVP Goal:** Build a working song-discovery system that recommends musically relevant but less-obvious tracks using both audio characteristics and historical listening patterns.

Today's Task Specification: Audio Similarity Retrieval using HNSW (ANN)

Target Dataset: backend/data/Train-Test Splits/Sequence/train_SEQ.csv

11 Audio Feature Dimensions: danceability, energy, valence, tempo, loudness, acousticness, instrumentalness, liveness, speechiness, key, mode.

Input Query Payload (Cleaned & Normalized):

```
{
"danceability": 0.9802, "energy": 0.9793, "valence": 0.5625,
"bpm": 116.69, "loudness_db": -20.82, "acousticness": 0.0071,
"instrumentalness": 0.1396, "liveness": 0.0, "speechiness": 0.0,
"key_int": 0, "mode": 1
}
Field Mapping: bpm -> tempo, loudness_db -> loudness, key_int -> key
```

Core Objectives:

- 1. Load and deduplicate train_SEQ.csv by track_id.
- 2. Normalize all 11 features to [0, 1] using MinMaxScaler.
- 3. Build a vector graph using HNSW (faiss.IndexHNSWFlat or hnswlib).
- 4. Perform Approximate Nearest Neighbor (ANN) search to return Top 5 most similar tracks with track IDs and Cosine Similarity scores.